

Idris Sadiq

Senior Software Engineer | React Node (Express/NestJS) AWS/Azure
Idrissdev@outlook.com | +1 (424) 458-6548 | Spring Valley, California, United States
<https://www.linkedin.com/in/idris-sadiq-1b824b3b4>
US citizen | White | English (Native)

PROFESSIONAL SUMMARY

Senior Software Engineer with **11+ years** building production web platforms across **enterprise software, SaaS, and digital product engineering**, specializing in **React v16–v18, Node.js v12–v20, TypeScript v4–v5, Next.js v12–v14, Express.js, PostgreSQL, Redis, AWS, and CI/CD** in one delivery-focused stack. Strong record modernizing legacy systems, fixing high-impact performance issues, and shipping secure APIs and scalable UI architecture with measurable gains in reliability, release speed, and user experience. Known for turning unclear requirements into stable features, reducing operational noise, and solving bugs, migration risks, and production incidents with practical engineering discipline.

SKILLS

- **Frontend** — **React v16–v18, Next.js v12–v14, TypeScript v4–v5**, JavaScript ES6+, Redux Toolkit, Zustand, React Query, HTML5, CSS3, Tailwind CSS, Styled Components, Webpack, Vite, Storybook
- **Backend & APIs** — **Node.js v12–v20, Express.js**, NestJS patterns, REST APIs, GraphQL, WebSockets, background jobs, JWT authentication, OAuth2, RBAC, OpenAPI/Swagger, API versioning, idempotency, caching strategies
- **Data, Messaging & Search** — **PostgreSQL**, MySQL, SQL Server, Redis, MongoDB, DynamoDB, Kafka, RabbitMQ, AWS SQS/SNS, Elasticsearch, OpenSearch
- **Cloud & DevOps** — **AWS** (EC2, ECS, EKS, Lambda, API Gateway, S3, RDS, CloudFront, IAM, CloudWatch), Azure, Docker, Kubernetes, Helm, Terraform, GitHub Actions, GitLab CI/CD, Nginx
- **Observability, Quality & Security** — Jest, React Testing Library, Cypress, Playwright, Supertest, Pact-style contract testing, OpenTelemetry, Datadog, Prometheus, Grafana, Sentry, structured logging, rate limiting, audit logging

EXPERIENCE

Senior Software Engineer

SumatoSoft | January 2021 - December 2025

- Led development of large-scale **React v18** and **Node.js v18** applications for multi-tenant SaaS products, shaping frontend architecture, API contracts, and deployment workflows that improved delivery consistency and reduced regression-heavy releases across multiple product teams.
- Built complex user-facing modules with **TypeScript, Next.js**, and reusable component patterns that improved dashboard responsiveness by **34%**, while making feature development faster through shared UI primitives, clearer state boundaries, and better frontend code organization.
- Designed and implemented secure **Node.js** service layers with **Express.js, PostgreSQL**, and **Redis**, creating cleaner domain separation and more reliable API behavior for high-volume business workflows that previously suffered from inconsistent validation paths.
- Resolved a recurring production issue where duplicate transaction submissions created data mismatches by introducing **idempotency keys**, database constraints, and retry-safe backend handlers, which reduced duplicate-processing incidents by **91%** and improved support confidence.
- Modernized legacy frontend screens from older class-based patterns into **React Hooks** and typed component structures, reducing maintenance friction and cutting average UI defect turnaround time by **29%** during ongoing feature delivery cycles.
- Implemented **React Query** caching, optimistic updates, and background refetch strategies to stabilize data-heavy screens, which reduced unnecessary API traffic by **38%** and improved perceived performance for internal operators handling large record sets.

- Migrated backend services from older **Node.js** runtime versions to **Node.js v18–v20** using staged compatibility checks, dependency audits, and CI validation, preventing release disruption while improving runtime stability and developer confidence.
- Introduced **OpenAPI**-driven contract validation and automated integration testing into CI/CD pipelines, allowing frontend and backend teams to work in parallel with fewer environment surprises and noticeably cleaner release handoffs.
- Diagnosed a memory growth issue caused by stale listeners and oversized in-process caches, then fixed it through lifecycle cleanup, cache policy changes, and profiling-guided refactoring that lowered restart frequency by **43%** in production.
- Implemented real-time user notifications with **WebSockets** and resilient reconnect handling, enabling faster operational feedback while preventing message loss scenarios that had previously caused inconsistent status visibility during peak usage windows.
- Strengthened application security with **JWT**, **OAuth2**, **RBAC**, audit logging, and request throttling, giving enterprise customers better control over role-sensitive actions while reducing risk in high-access administrative workflows.
- Partnered closely with product and QA teams to deliver new features, bug fixes, and rollout plans through feature flags and safer deploy practices, consistently balancing speed with stability across active customer-facing releases.

Senior Software Engineer

High Touch Technologies | June 2018 - December 2020

- Developed end-to-end business applications using **React v16–v17**, **Node.js v14–v16**, and **TypeScript**, helping modernize core product workflows while preserving compatibility with existing integrations and historically sensitive customer processes.
- Built reusable frontend architecture for forms, reporting pages, and administrative dashboards, which improved implementation consistency and cut repeated UI development effort by **31%** across several parallel delivery streams.
- Created **REST APIs** and backend modules with **Node.js**, **Express.js**, and **PostgreSQL**, improving service reliability through stronger validation, clearer error contracts, and cleaner data-handling logic for operationally critical flows.
- Solved a long-standing issue where slow database queries caused intermittent request timeouts by introducing indexing improvements, query rewrites, and targeted caching, lowering average response latency by **46%** on the most-used endpoints.
- Led a gradual migration from tightly coupled legacy frontend code into a more maintainable **React** component model, allowing teams to deliver new features incrementally instead of taking on a risky full-platform rewrite.
- Improved frontend rendering efficiency on data-heavy pages by reducing unnecessary re-renders, memoizing expensive view logic, and refining client-side state updates, which increased UI responsiveness by **28%** during peak operator usage.
- Implemented file import, export, and background processing workflows using queue-based patterns and fault-tolerant job handling, giving support teams better traceability when malformed files or partial processing failures occurred.
- Reduced recurring release issues by restructuring CI/CD steps, pinning build dependencies, and separating unit, integration, and end-to-end checks, which improved deployment success rates by **24%** over earlier delivery baselines.
- Added structured logging, centralized error capture, and dashboard visibility with **CloudWatch** and **Datadog**, making it easier to trace incidents back to specific requests, code paths, and failing dependencies under real production load.
- Worked closely with stakeholders to implement new customer-facing features while also addressing high-priority bugs, ensuring that technical debt reduction stayed aligned with business value rather than becoming a disconnected cleanup effort.
- Supported cloud deployment and runtime hardening across **AWS** services and containerized environments, helping teams ship more confidently through better environment parity, clearer rollback procedures, and safer release packaging.

Software Engineer

Centric Tech Inc. | September 2015 - May 2018

- Built internal and client-facing web applications using **React v15–v16**, **Node.js v8–v10**, JavaScript, and SQL-backed services, contributing to practical feature delivery during a period when frontend architecture was becoming more component-driven.
- Developed CRUD-heavy workflow modules, reporting interfaces, and backend endpoints that improved business process efficiency, while establishing stronger alignment between frontend behavior and backend validation for multi-step user actions.
- Contributed to early modernization efforts by introducing reusable **React** components into older application areas, making selected screens more interactive without forcing a full rewrite of stable but aging platform modules.
- Resolved an issue where concurrent edits caused silent data overwrites by adding safer update checks, clearer user feedback, and stronger backend guardrails, which significantly reduced reconciliation work for operations teams.
- Improved API and page performance through query tuning, index changes, and payload cleanup, producing a **27%** improvement in response times for several high-traffic administrative functions used throughout the workday.
- Enhanced backend error handling and request tracing so that failed jobs and malformed submissions produced more actionable diagnostics, giving engineers faster root-cause analysis during defect triage and on-call support.
- Implemented authentication and session management improvements that reduced permission-related inconsistencies, while aligning application behavior more closely with role-specific access expectations in enterprise-facing deployments.
- Helped introduce automated test coverage for core frontend and backend paths, increasing release confidence and reducing the frequency of last-minute bug discoveries during manual QA and staging validation cycles.
- Worked across frontend, backend, and deployment tasks with flexibility, regularly balancing new feature delivery, issue investigation, and technical cleanup to keep legacy systems stable while extending product capability.

Software Engineer Intern

Microsoft | September 2014 - August 2015

- Supported internal web tooling initiatives using JavaScript, early **Node.js**, API integration patterns, and component-based frontend development practices that strengthened engineering fundamentals in scalable product delivery environments.
- Contributed to internal dashboard enhancements and service-connected UI behavior, gaining hands-on experience with maintainable frontend structure, backend communication patterns, and code review expectations in a large engineering organization.
- Investigated defects related to inconsistent data presentation and asynchronous request handling, then helped deliver fixes that improved usability, reduced confusion for internal users, and strengthened trust in operational dashboards.
- Assisted with build, test, and deployment automation tasks that exposed the importance of repeatable engineering workflows, especially when multiple contributors were changing frontend assets and backend-connected behavior at the same time.
- Worked with senior engineers to understand practical approaches to debugging, service integration, and incremental feature delivery, building a strong foundation in production-minded development that shaped later full-stack engineering growth.
- Helped document implementation details, validation behavior, and known issues for shared tooling modules, improving handoff quality and making future maintenance easier for engineers extending the same internal systems.
- Participated in code reviews and bug triage discussions, learning how structured feedback, reproducible issue reports, and careful testing habits directly improve code quality and delivery confidence in real teams.
- Gained early exposure to performance thinking by profiling slow UI paths and inefficient request patterns, then applying small but meaningful fixes that improved responsiveness in frequently used internal screens.

- Built a practical understanding of how frontend logic, backend endpoints, and deployment processes fit together, which created a durable foundation for later ownership of full-stack **React** and **Node.js** production systems.

EDUCATION

Harvard University

Bachelor's degree in Computer Science

2010 - 2014

- Completed a rigorous CS curriculum with strong foundations in **systems**, **algorithms**, and **software engineering**, applying these principles to production-grade web and backend development through team-based projects and practical coursework.